

Scalable Market Basket Analysis & Product Cross-Selling System

Project Authors:	Tanaya Salunke & Sankhya Londhe	Academic Context:	Honors Computer Science Senior Capstone
Primary Dataset:	Online Retail II (1,067,371 Raw Records)	Core Stack:	Python, SciPy, PySpark, Streamlit, Plotly

1. Executive Summary & Project Background

In modern retail e-commerce, identifying customer purchasing patterns from large transactional datasets is essential for optimizing product cross-selling, promotional bundling, and shelf arrangement. This project presents an enterprise-grade, end-to-end **Market Basket Analysis (MBA)** and Product Cross-Selling system developed by **Tanaya Salunke** and **Sankhya Londhe**. Utilizing the benchmark **Online Retail II dataset** (1,067,371 transaction rows across 2009–2011), the platform ingests raw sales logs, executes data hygiene, encodes transactions using memory-efficient sparse boolean matrices (SciPy CSR), benchmarks pattern mining algorithms (**Apriori vs. FP-Growth**), prunes misleading spurious rules, provides Explainable AI (XAI) feature attribution, and serves real-time recommendations via a role-authenticated web app.

1.1 Key Operational Challenges Addressed

- **Memory Bottlenecks:** Standard multi-hot binary matrices consume over 210 MB RAM for 39,517 baskets across 5,321 products. Our SciPy CSR sparse matrix format reduces memory footprint to 4.72 MB (97.7% reduction).
- **Algorithmic Candidate Explosion:** Traditional Apriori requires k database passes and suffers candidate explosion (C_k). FP-Growth constructs a compact FP-Tree in 2 passes with zero candidate generation, running up to 20x faster.
- **Spurious Association Rules:** High confidence rules involving universally popular products (e.g. Toothpaste → Bread) are often uninformative. Automated filtering of rules with Lift < 1.2 eliminates misleading bundles.

2. System Architecture & Module Pipeline

The system architecture is organized into seven decoupled, sequential processing layers designed for modular execution, testability, and big-data scalability:

Data Ingestion & Hygiene Cleans 1.06M rows (strips returns, zero prices)	→	CSR Sparse Matrix Encoder Groups baskets into 4.72 MB boolean matrix	→	FP-Growth Benchmarking 20x faster frequent itemset mining engine
Streamlit Web UI & XAI Role Auth app with Plotly XAI visualizers	■	Out-of-Core Batching Handles 50k-row chunks & PySpark MLlib module	■	Association Rule Filter Computes Lift/Confidence; prunes Lift < 1.2

Pipeline Layer	Technical File	Functional Role & System Transformation
Stage 1: Ingestion	src/data_loader.py	Filters return invoices ('C'), zero prices, non-product fees. Output: 1,036,154 clean rows.
Stage 2: Encoding	src/encoder.py	Groups baskets into SciPy csr_matrix boolean format. Matrix sparsity = 99.53%.
Stage 3: Benchmark	src/frequent_mining.py	Evaluates Apriori vs FP-Growth across min_support thresholds (0.015 to 0.05).
Stage 4: Rule Pruning	src/rule_generator.py	Computes Support, Confidence, Lift, Leverage, Conviction. Prunes spurious Lift < 1.2 rules.
Stage 5: XAI & Graph	src/xai_explainer.py src/network_graph.py	Computes XAI confidence attribution waterfall and renders NetworkX co-occurrence graphs.
Stage 6: Scalability	src/scalability.py src/pyspark_fpgrowth.py	Out-of-core 50k-row chunk stream processor & PySpark MLlib distributed cluster mapping.
Stage 7: Web App	app/streamlit_app.py	Dark neon glassmorphism UI with Admin/User login, 3D Plotly visualizers & cross-sell lookup.

3. Detailed Technology Stack & Component Specifications

The project leverages a modern Python data science and enterprise engineering stack. Every tool was chosen to ensure optimal computational performance, analytical rigor, and production-grade usability:

Technology Layer	Component & Version	Technical Rationale & Architectural Purpose
Core Language	Python 3.9	Primary language environment providing robust scientific and statistical library support.
Sparse Matrix Engine	SciPy 1.13 (csr_matrix)	Compresses 210 MB transaction matrix to 4.72 MB by storing only non-zero purchase indices.
Data Processing	Pandas 2.3, NumPy 2.0, PyArrow	Handles chunked ingestion (50k rows), vector mathematical transformations, and Parquet caching.
Pattern Mining	MLxtend 0.23, PyFPGrowth 1.0	Executes Apriori level-wise candidate mining and FP-Growth FP-Tree pattern extraction.
Distributed Computing	Apache Spark 3.5 / PySpark	Distributed big-data reference architecture (pyspark.ml.fpm.FPGrowth) for multi-node clusters.
Explainable AI (XAI)	Custom XAI Attribution Engine	Decomposes rule confidence into Baseline Consequent Support P(Y) vs Lift Multipliers.
Visualizations	Plotly 5.22, NetworkX 3.2	Generates 3D Scatter Topology maps, XAI Waterfall charts, and NetworkX force-directed graphs.
Interactive Web UI	Streamlit 1.50 (Custom CSS)	Delivers Dark Neon Glassmorphism dashboard with role authentication (User/Admin).
Testing & QA	Pytest 8.4	Automated unit test suite validating hygiene, encoding, mining, and rule generation logic.
Document Engine	ReportLab 4.2	Programmatic PDF generation engine for executive proposal and final report generation.

4. Theoretical Methodology & Algorithmic Mechanics

4.1 Mathematical Formulation of Association Rule Metrics

Given a set of transactions T and itemsets A (antecedent) and C (consequent):

- **Support:** Joint probability of buying both A and C: $Support(A \rightarrow C) = P(A \cap C) = Count(A \cup C) / |T|$
- **Confidence:** Conditional probability of buying C given A: $Confidence(A \rightarrow C) = P(C | A) = Support(A \cup C) / Support(A)$
- **Lift Ratio:** Multiplicative boost over independent chance: $Lift(A \rightarrow C) = P(A \cap C) / (P(A) \cdot P(C)) = Confidence(A \rightarrow C) / Support(C)$
- **Leverage:** Absolute difference between joint support and expected support: $Leverage(A \rightarrow C) = Support(A \cup C) - Support(A) \cdot Support(C)$
- **Conviction:** Ratio of expected incorrect prediction to observed incorrect prediction: $Conviction(A \rightarrow C) = (1 - Support(C)) / (1 - Confidence(A \rightarrow C))$

4.2 Mechanical Comparison: Apriori vs. FP-Growth

Algorithmic Metric	Apriori Algorithm (Level-Wise)	FP-Growth Algorithm (Pattern-Tree)
Database Passes	Requires k full scans for itemsets of length k.	Exactly 2 scans (Pass 1: Frequency, Pass 2: Tree).
Candidate Generation	Generates combinatorial candidates (C_k).	Zero candidate generation (Direct FP-Tree mining).
Data Structure	Flat hash-trees and array combinations.	Compact in-memory FP-Tree (Trie structure) .
Runtime (Supp = 0.015)	18.42 seconds (High CPU overhead).	1.12 seconds (16.4x speedup) .
Peak RAM Footprint	142 MB (Candidate itemset buffer).	18.4 MB (Compact tree memory footprint) .

5. Detailed Implementation Strategy & File Structure

The project source code is fully implemented across clean Python scripts in src/ and app/, backed by automated unit tests:

File Path	Implementation Details & Methods	Verification & Output Artifact
src/data_loader.py	Ingests Excel/CSV; strips returns (C), non-product fees, negative quantities.	1,036,154 clean rows in data/processed/cleaned_transactions.csv
src/encoder.py	Groups items by invoice into csr_matrix. Saves item mapping dictionary.	SciPy sparse array (39,517 × 5,321; 4.72 MB RAM footprint)
src/frequent_mining.py	Runs Apriori & FP-Growth across support levels; logs runtime and memory.	Benchmark tables in reports/benchmarks/algo_comparison.csv
src/rule_generator.py	Extracts rules; filters spurious Lift < 1.2 associations; ranks by Lift.	Clean rule set saved to reports/rules/pruned_rules.csv
src/xai_explainer.py	Decomposes confidence into baseline P(Y) vs Lift boost using Plotly Waterfall.	Interactive XAI chart rendered in Streamlit UI
src/network_graph.py	Constructs NetworkX co-occurrence graph with force-directed layout.	Interactive graph plot & PNG export in reports/insights/
src/scalability.py	Out-of-core batch processor (50,000-row streaming chunks) & StageProfiler.	Batch statistics covering 5.8M product pairs under 800MB RAM
app/streamlit_app.py	Dark neon UI with Admin/User auth, 3D Plotly rule topology, & cross-sell tool.	Interactive web application running on http://localhost:8501
tests/test_pipeline.py	Pytest automated unit test suite covering hygiene, encoding, mining, rules.	100% passing automated test suite (4/4 tests passed)

6. Work Plan & Phase-Wise Milestones

- **Phase 1 (W1-W2):** Environment setup, project schema creation, dependency locking, dataset acquisition.
- **Phase 2 (W3-W4):** Ingestion engine & data hygiene implementation (return filtering, invoice normalization).
- **Phase 3 (W5-W6):** Sparse CSR boolean matrix encoder implementation & basket density EDA profiling.
- **Phase 4 (W7-W8):** Apriori vs FP-Growth benchmarking across support thresholds (0.015 to 0.05).
- **Phase 5 (W9-W10):** Association rule mining, metric calculation, and spurious rule filtering (Lift < 1.2).
- **Phase 6 (W11-W12):** XAI feature attribution engine, NetworkX co-occurrence graph, and regional segment mining.
- **Phase 7 (W13-W14):** Out-of-core streaming batch engine, PySpark MLlib module, and Streamlit dark neon UI with auth.
- **Phase 8 (W15-W16):** Pytest suite execution, automated PDF document generation, and GitHub repository synchronization.

7. Risk Management & Academic References

7.1 Risk Mitigation Strategy

- **Risk A: Memory Exhaustion at Low Support Thresholds.** *Mitigation:* SciPy CSR sparse matrix encoding and max_len=4 candidate constraints limit peak RAM footprint under 20 MB.
- **Risk B: High-Confidence Spurious Bundling.** *Mitigation:* Automated pruning of rules with Lift < 1.2 ensures only true synergistic product pairs are surfaced.

7.2 Key Academic References

1. Agrawal, R., & Srikant, R. (1994). *Fast Algorithms for Mining Association Rules in Large Databases*. Proc. 20th VLDB, 487–499.

2. Han, J., Pei, J., & Yin, Y. (2000). *Mining Frequent Patterns without Candidate Generation*. ACM SIGMOD Record, 29(2), 1–12.

3. Chen, D., Sain, S. L., & Guo, K. (2012). *Data mining for the online retail industry*. Journal of Database Marketing, 19(3), 197–208.

4. Raschka, S. (2018). *MLxtend: Machine learning data science utilities*. Journal of Open Source Software, 3(24), 638.